

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт
про виконання лабораторної роботи №N1
«Повторення роботи з С»

Виконав
студент групи Феп-12
Носов А.О.

Львів-2026

Мета роботи

Пригадати основи програмування мовою C. Навчитися створювати власні типи даних (структури) в C++, розбивати великий код на кілька окремих файлів (заголовні та основні), а також закріпити вміння зчитувати дані звичайних текстових документів.

1. Хід роботи
Створив структуру Money, яка складається з двох полів: grn (ціле число) та kor (коротке ціле).
2. Розділив програму на файли: створив заголовний файл Money.h, де описав саму структуру та назви всіх математичних функцій.
3. У файлі Money.cpp написав логіку функцій: правильне перенесення надлишку копійок у гривні (нормалізацію), додавання двох сум, множення ціни на кількість та заокруглення копійок до найближчого десятка.
4. Створив текстовий файл data.txt із тестовими даними (ціна в гривнях, копійках та кількість товару).
5. Написав функцію для зчитування даних з файлу, яка по черзі дістає кожен товар, рахує його загальну вартість та додає до загального чеку.
6. У файлі main.cpp залишив лише підключення свого заголовного файлу та виклик функції обчислення.
7. Скомпілював програму, перевірів правильність роботи математичних функцій та заокруглення на прикладі з файлу.

```

1  #include "Money.h"
2  #include <fstream>
3
4  using namespace std;
5
6  void normalize(Money& m) {
7      if (m.kop >= 100) {
8          m.grn += m.kop / 100;
9          m.kop = m.kop % 100;
10     }
11     if (m.kop < 0) {
12         m.grn -= 1;
13         m.kop += 100;
14     }
15 }
16
17 int add(const Money& a, const Money& b, Money& result) {
18     result.grn = a.grn + b.grn;
19     result.kop = a.kop + b.kop;
20     normalize(result);
21     return 0;
22 }
23
24 int multiply(Money& price, int quantity) {
25     price.grn *= quantity;
26     price.kop *= quantity;
27     normalize(price);
28     return 0;
29 }
30
31 int roundMoney(const Money& m, Money& result) {
32     int lastDigit = m.kop % 10;
33
34     if (lastDigit < 5)
35         result.kop = m.kop - lastDigit;
36     else
37         result.kop = m.kop - lastDigit + 10;
38
39     result.grn = m.grn;
40     normalize(result);
41     return 0;
42 }
43
44 void printMoney(const Money& m, const string& label) {
45     if (!label.empty())
46         cout << label;
47     cout << m.grn << " UAH ";
48     if (m.kop < 10)
49         cout << "0";

```

Демонстрація роботи бібліотеки

```

#include "Money.h"

int main() {
    processFile("data.txt");
    return 0;
}

```

Вместимо файлу мейн

```

artno@vivobook MINGW64 ~/OneDrive - lnu.edu.ua/OOP/lab1 (main)
● $ bash run.sh
--- Items ---
Item 1: price = 19 UAH 89 kop
      quantity: 3
      cost:      59 UAH 67 kop

Item 2: price = 13 UAH 29 kop
      quantity: 1
      cost:      13 UAH 29 kop

--- Result ---
Total: 72 UAH 96 kop
To pay (after rounding): 73 UAH 00 kop

artno@vivobook MINGW64 ~/OneDrive - lnu.edu.ua/OOP/lab1 (main)
○ $ 

```

Вивід програми

Висновок

У цій лабораторній роботі я написав програму, яка рахує загальну вартість покупок за даними з текстового файлу. Для цього я створив структуру Money, яка окремо зберігає гривні та копійки.

Щоб код не був суцільним шматком тексту, я розділив його: у головному файлі залишив тільки запуск, а всі математичні дії виніс в окремий файл. Я написав функції, які вмiють правильно додавати гроші, множити ціну на кількість, переносити зайві копійки в гривні (якщо їх більше ста) та заокруглювати залишок. Також на практиці спробував працювати з посиланнями (&), завдяки яким функції змогли оновлювати оригінальні змінні без створення зайвих копій.

